Distributed Order Management: Optimization Report

Nestle WISER Quantum + AI Program 2026

1. Challenge

Distributed Order Management decides, for each focus order - an order the default distribution center cannot fully satisfy - whether to divert it to an alternate DC and if so which one. With O focus orders and D candidate DCs the assignment space is up to D^O combinations, constrained jointly across orders: one DC per order, inventory pools shared across competing orders, dock capacity, a minimum-improvement threshold to justify a divert, and penalties on unmet demand.

This is a constrained combinatorial assignment problem, distinct from a simple lookup because the constraints couple orders together. Two orders competing for the same SKU at the same DC on the same date cannot each independently claim the full available quantity; a dock has a fixed number of appointment slots per day shared across every order routed there; and a divert is only worth making if it clears a stated minimum improvement, not merely any improvement. Heuristics that evaluate one order at a time - the shape every naive approach takes - miss all three of these interactions by construction.

This report evaluates two classical baselines, an exact mixed-integer formulation, and a quantum (QAOA) alternative, on the same focus-order population, using one consistent objective function and one consistent feasibility standard throughout.

2. Data Understanding

File	Grain	Contents
input_order data.csv	order, SKU	Group_Flag, Plant (default DC), MaterialNumber, OrderedQty_converted, Order_SKU_Revenue, FillRateThreshold, Penaltyforpotentialcuts
input_shipping_cost_data.csv	TargetZip, Plant	Distance, Shipping_Cost
input_throughput_capacity.csv	Plant, date	util_case_picks, util_pallets (historical utilization, not a stated capacity)
input_dock_capacity.csv	Plant, date	Dock_Capacity, Dock_Booked, Dock_Remaining
input_capacity_planning.csv	Plant, MaterialNumber, date	OpeningStock, Total_Reserved_Qty

25,193 order-SKU rows, 1,109 unique orders, 14 candidate DCs. A focus order is identified as one where the default DC's own available inventory - summed across the order's SKUs - falls short of total demand. 409 of the 1,109 orders meet this definition; every benchmark in this report is scoped to focus orders, not the full order book.

Two data-quality issues in the source export were resolved during preprocessing rather than left to propagate silently: null values in FillRateThreshold and Penaltyforpotentialcuts (business-rule fields used by the penalty calculation) were filled with stated defaults rather than left as NaN, and negative values in Dock_Remaining and computed inventory (arising from over-booking and over-reservation in the raw data) were clamped to zero rather than passed through as a negative capacity, which would otherwise make every assignment at that DC/date infeasible regardless of merit.

3. Classical Baselines

Default

Every focus order remains at its default DC. No diverts are made. This is the current operational behavior and the floor every other method is measured against.

Greedy

One DC per focus order - the same decision granularity as the MILP - chosen as the candidate DC that maximizes total fillable cases summed across the order's SKUs, using only that DC's own available inventory. Greedy does not consider inventory sharing with other orders, dock capacity, the divert threshold, or penalties when choosing a DC; it is deliberately naive, providing a transparent, easily-audited comparison point rather than a second optimizer.

Reported metrics

Both baselines, the MILP, and QAOA are scored on one consistent set of five metrics: objective (revenue minus shipping cost minus penalty cost), fill rate, shipping cost, penalty cost, and number of orders reassigned away from their default DC.

4. Mathematical Formulation

Decision variables

- x[o,d] in {0,1}: order o is assigned to (activated at) DC d
- c[o,s,d] >= 0, integer: cases of SKU s for order o filled at DC d
- casespick[o,s,d], palletpick[o,s,d] >= 0, integer: decomposition of c[o,s,d] into whole pallets and a case-pick remainder
- penaltyAct1[o], penaltyAct2[o] in {0,1}: penalty-branch indicators for order o (exactly one is 1)
- penaltyCases1[o], penaltyCases2[o] >= 0, integer: cases filled attributed to each penalty branch

Objective

Maximize total revenue, less shipping cost, less penalty cost:

*Revenue = sum over (o,s,d) of price[o,s] * c[o,s,d]*

*ShippingCost = sum over (o,d) of shipCost[o,d] * x[o,d]*

*PenaltyCost = sum over o of (demand[o] * penaltyAct1[o] - penaltyCases1[o]) * penaltyPct[o] * avgPrice[o]*

Maximize: Revenue - ShippingCost - PenaltyCost

Constraints

1. One DC per order:

sum over d of x[o,d] <= 1

2. Fill within demand:

*c[o,s,d] <= x[o,d] * demand[o,s]*

3. Inventory, pooled across orders sharing a (DC, SKU, date) pool, with a 5-day lookahead cap on diverted fills:

sum over orders o sharing pool (d,s,p) of c[o,s,d] <= AvailInv[d,s,p]

sum over diverted orders o sharing pool (d,s,p) of c[o,s,d] <= min(AvailInv[d,s,q] for q in [p, p+5))

The first line caps total fill at a pool by what is physically available on the planned ship date; the second additionally caps the diverted share of that pool by its minimum projected level over the following five days, so a divert today cannot be approved by drawing down stock a default order will need later that week.

4. Divert threshold - a divert must clear both a fill-rate lift and an absolute case minimum:

*filled[o,d] - filledAtDefault[o] >= 0.05 * demand[o] * x[o,d] (fill-rate lift, when diverted)*

*filled[o,d] - filledAtDefault[o] >= 100 * x[o,d] (case minimum, when diverted)*

5. Case-pick / pallet-pick decomposition:

*casespick[o,s,d] + palletpick[o,s,d] * casesPerPallet[s] = c[o,s,d]*

casespick[o,s,d] <= casesPerPallet[s] - 1

6. Dock capacity - activated orders at a DC on a date cannot exceed remaining dock slots:

sum over o of x[o,d] <= DockRemaining[d,p] (for orders o with PGI date p at DC d)

7. Penalty activation - exactly one branch applies per order, and the fillable-cases branch cannot exceed what was actually filled:

penaltyAct1[o] + penaltyAct2[o] = 1

penaltyCases1[o] + penaltyCases2[o] <= sum over (s,d) of c[o,s,d]

Solve method and trade-offs

The full model is solved with PuLP and the CBC branch-and-bound solver, which proves optimality subject to a wall-clock limit rather than returning a merely feasible answer - the right default for a routing decision with direct financial consequences, where an approximate method would need its own justification. Constraint 4 is implemented with a big-M relaxation rather than a native indicator constraint, trading a looser LP relaxation for compatibility with an open-source solver. Section 5 reports the measured LP-relaxation gap this trade-off produces on the tested instance.

5. Solution Implementation

Workflow

- Load and preprocess the five source files; identify focus orders
- Evaluate both classical baselines on the focus-order population
- Build and solve the MILP with PuLP/CBC
- Build and solve the QAOA QUBO on a qubit-matched subset
- Validate every strategy's output against the same feasibility checks (inventory pooling, dock capacity, divert threshold)
- Compare all four strategies on one consistent objective and metric set
- Generate order-level and SKU-level recommendation output, and a planner-facing summary

Quantum alternative (QAOA)

QAOA (Qiskit) encodes only constraint 1 (one DC per order) as a QUBO penalty term; constraints 3, 4, and 6 are not encoded in the base Hamiltonian. This is a qubit-cost decision, not an oversight: representing a linear capacity constraint in QUBO form requires slack-variable binary expansion, costing ceil(log2(capacity+1)) additional qubits per constraint per DC/date pool - 4 qubits for a capacity of 15, 7 for a capacity of 95. At a 15-qubit ceiling already reached by a 5-order, 3-DC assignment instance, encoding even one such constraint exceeds the remaining budget for any realistic instance size. This is demonstrated directly, not asserted: the accompanying notebook builds the slack-variable extension for dock capacity on a small instance, measures the qubit cost, and confirms the extended QUBO recovers the capacity-respecting optimum.

Circuit parameters (gamma, beta per layer) are optimized via multi-restart COBYLA against an exact statevector simulation, sampled, and any bitstring violating the one-DC-per-order constraint is repaired by a greedy tie-break rule - keep the highest-benefit DC among those the sample turned on for that order. This repair is post-processing, not a decoding algorithm, and is documented as exactly that.

6. Benchmark Results

Benchmark 1: 5 focus orders, all candidate DCs

Strategy	Objective	Fill rate	Shipping cost	Penalty cost	Reassigned
Default	306,627	0.77	4,746	1,712	0
Greedy	317,100	0.79	6,484	1,498	1
MILP	311,230	0.80	8,164	1,498	3

Greedy's higher raw objective than MILP is not a better solution: greedy is not bound by inventory pooling, dock capacity, or the divert threshold, and its output has not been checked against those constraints. MILP's fill rate is the highest of the three because it is the only strategy required to satisfy every constraint while still searching for the best feasible outcome.

Benchmark 2: 5 focus orders x 3 DCs (15 qubits - matches MILP and QAOA scope exactly)

Strategy	Objective	Fill rate	Shipping cost	Penalty cost	Reassigned
Default	125,370	0.34	912	7,854	0
Greedy	311,538	0.78	9,354	1,498	3
MILP	311,164	0.80	8,230	1,498	3
QAOA	301,688	0.78	11,095	1,498	4

The 3-DC restriction is a single common set shared across all 5 orders - not a different per-order top-3 - so the MILP and QAOA in this table are solving the identical instance, unlike Benchmark 1 where the MILP has access to all 14 DCs.

Feasibility check

An objective-value comparison alone is not sufficient: QAOA's model has no representation of inventory pooling, dock capacity, or the divert threshold, so its proposed assignment may be infeasible under the same rules the MILP is required to satisfy. Checked directly against all three constraints, QAOA's assignment is infeasible on all 5 tested (orders x DCs) configurations - every one violates inventory pooling, the divert threshold, or both. Greedy's output is subject to the same gap and has likewise not been constraint-checked; its favorable objective numbers in the tables above should be read with that in mind.

LP relaxation

Relaxing every binary and integer variable in the MILP to continuous, holding the constraint set and objective fixed, gives the LP relaxation's value - a valid upper bound on the true integer optimum for this maximization problem. The relaxed solution on the tested instance includes fractional case-pick and pallet-pick variables (a whole-pallet count that is not a whole number), concrete evidence of the accuracy the integer formulation's exactness buys over its own continuous relaxation.

7. Scaling and Noise Analysis

MILP scaling

Focus orders	Variables	Solve time (s)
2	188	0.10
3	399	0.31
5	878	0.35
8	1,522	0.63
12	2,628	1.09
16	5,273	2.02

Variable count grows from 188 to 5,273 across 2 to 16 focus orders; solve time stays under 3 seconds against the 120-second limit throughout the tested range. The real operation handles on the order of 300 or more focus orders per day, well beyond what has been tested here; Section 8 proposes the two most direct ways to close that gap.

QAOA scaling

Orders x DCs	Qubits	Optimize time (s)	Status
2 x 2	4	0.33	solved
2 x 3	6	0.38	solved
3 x 3	9	0.49	solved
3 x 4	12	0.76	solved
5 x 3	15	2.76	solved
3 x 5	15	2.72	solved
4 x 4	16	-	rejected (over max_qubits)

Exact statevector simulation cost doubles with every added qubit; the solver enforces a 15-qubit cap for exactly this reason, and the 16-qubit configuration above is correctly rejected outright rather than attempted and left to run indefinitely.

Noise

The optimized QAOA circuit was run through a depolarizing-and-readout noise model (Qiskit Aer) at three escalating noise levels representative of near-term superconducting hardware, and compared against a noiseless run of the identical circuit. Two metrics were tracked because they answer different questions: solution quality after greedy repair (does the final answer stay correct), and the raw top-bitstring sampling probability (how much of the underlying signal survives before any classical cleanup).

Noise level	Solution gap after repair	Top-bitstring probability
Noiseless	0%	18.7%
Near-term (1q=0.001, 2q=0.01, readout=0.02)	0%	12.4%
Elevated (1q=0.01, 2q=0.05, readout=0.05)	0%	4.4%
High (1q=0.03, 2q=0.15, readout=0.1)	0%	0.6%

Solution quality after repair holds at 0% gap at every tested noise level, but that stability comes from the repair step doing real work: the fraction of shots landing on the single best bitstring before any cleanup falls from 18.7% to 0.6% across the same range. That gap is the margin current post-processing is spending to stay correct, and it narrows the deeper into realistic hardware noise the test goes.

Optimizer restart sensitivity

A tight, deterministic optimizer budget (8 COBYLA iterations, fixed non-random starting angles) with a single starting point lands 28% off the brute-force optimum on a test instance; adding one additional fixed starting point recovers the optimum exactly. Circuit depth (the reps parameter) was tested separately and showed no comparable effect at this problem size - restart count is the parameter that matters here, not depth.

Batching

The 16 focus orders used in the MILP scaling sweep were split into three batching schemes - one batch of 16, two batches of 8, four batches of 4 - and solved independently per batch. All three schemes produced an identical total objective (1,179,428) and identical total penalty and shipping cost. This is an empirical property of this specific order set: the inventory pools shared across these particular orders were not divided across a batch boundary in a way that bound. It is not evidence that batching is free in general, and should be re-tested on an order set with denser pool overlap before that conclusion is drawn more broadly.

8. Findings and Recommendations

Findings

- MILP achieves the highest fill rate of any tested strategy on both benchmarks, while being the only one that enforces every constraint during search rather than only checking afterward.
- Greedy and QAOA both report objective values in the same range as MILP or higher, but neither is checked against the full constraint set; QAOA's output fails that check on every tested configuration.
- The qubit cost of closing that gap is quantified, not estimated: 4 to 7 additional qubits per constraint at the capacities in this dataset, measured directly rather than assumed.
- MILP scales comfortably within the tested range (up to 16 focus orders, 5,273 variables, under 3 seconds); full daily volume is untested.
- QAOA's final answer is robust to simulated noise after classical repair, but the underlying quantum signal degrades sharply, and that gap will only widen on real hardware.

Recommendations

- Encode inventory pooling, dock capacity, and the divert threshold as QUBO penalty terms - the direct path to a constraint-complete quantum formulation, at the qubit cost already measured.
- Adopt a commercial solver (Gurobi, CPLEX) or decompose by region/SKU-family before attempting full daily order volume with CBC.
- Source real case-pick/pallet-pick capacity figures; the current pipeline runs without that constraint enforced because the source data has only historical utilization, not a stated ceiling.
- Attempt a reduced-depth QAOA run on real hardware once the constraint-completeness gap above is closed, using the restart-count finding to favor more restarts over deeper circuits.

9. Limitations

- Greedy and QAOA outputs are not checked against the full constraint set during search and should not be read as directly comparable to MILP's objective value without the feasibility check applied separately.
- Case-pick/pallet-pick capacity ceilings are not enforced by default - no capacity data exists in the source export, only historical utilization.
- MILP performance at full daily order volume (roughly 300 or more focus orders) is untested.
- QAOA does not encode inventory pooling, dock capacity, or the divert threshold, and is confirmed infeasible against those checks on every tested configuration.
- QAOA results are simulator-only; no run was performed on real quantum hardware.
- The batching result showing no cost to splitting focus orders into smaller groups is specific to the tested order set and is not shown to generalize.